yarnnn

Autonomous agents that know your work.

They connect to your platforms. They accumulate knowledge. They work while you don't.

(Pre) Seed Round 2026

Kevin Kim · kvkthecreator@gmail.com

It looks like the big players will own everything.

Claude has coding agents, desktop, cowork.
ChatGPT has memory, Codex, Sora.
The market assumes they'll consume every layer.

But every platform cycle looks that way — until it doesn't.

Google didn't become Salesforce.
Facebook didn't become Shopify.
AWS didn't become Datadog.

The Honest Bet

History never repeats, but it does rhyme. Our thesis is that even with the AI tech revolution, the application layer is yet to come.

2008: Google	→	Looked like it would own everything on the web. It didn’t become Salesforce.
2012: Facebook	→	Looked like it would own all of social commerce. It didn’t become Shopify.
2015: AWS	→	Looked like it would own every cloud application. It didn’t become Datadog.
2026: LLMs	→	Look like they’ll own every layer. But code was the easy case. Work context is the hard one.

The application layer of the AI platform shift is yet to come, and we’re taking the **bold bet** that it will be Agent-native, and the work economy will shift from human -> agents

Proof of Demand

ClawdBot proved the appetite is real.

17,830

GitHub stars in 24 hours

Fastest single-day growth in GitHub history.

> *"The first time I've felt like I'm living in the future since ChatGPT launched."*
>
> **— ClawdBot user**

But 95% of people can't use it.

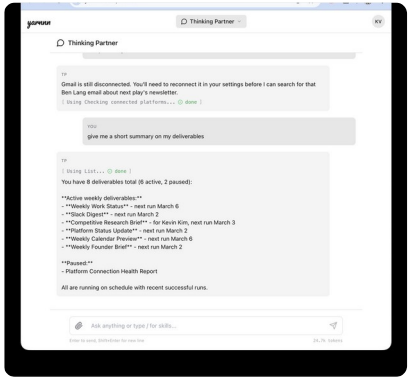
High technical barrier, privacy, hardware setup

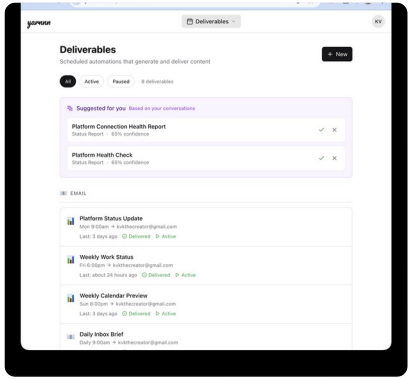
- ✘ Requires VPS provisioning (only 5% can do this)
- ✘ 200+ security leaks detected (GitGuardian)
- ✘ No project isolation or recurring workflows

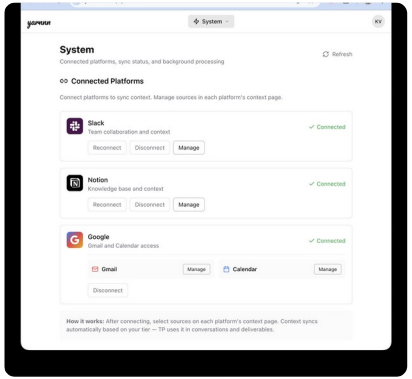
Product
Architecture
Solution / Moat

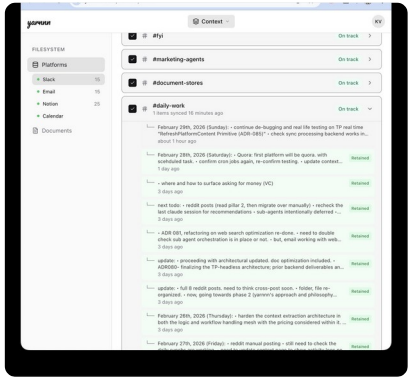
Product: Meet yarnnn's Orchestrator

Your autonomous agent network. Live at yarnnn.com.









T+0s Connect

Link Slack, Gmail, Notion, Calendar. Smart defaults. No configuration required.

T+60s Auto-Bootstrap

First agent created automatically. First output delivered within minutes.

Days Accumulate

Every sync deepens knowledge. Every edit refines quality.

Weeks Compose

System identifies new agents from your accumulated substrate.

Agent Architecture and Orchestration

Built for Agent-native, autonomous workflows

Every layer purpose-built so knowledge compounds — creating recursive feedback between agent output and intelligence

Perception Pipeline
Slack • Gmail
Notion • Calendar

→

Knowledge Base
Unified content
Retention + provenance

→

Memory
Workspace + feedback
Edit distillation

→

Agent Engine
5 execution modes
Chat + headless

→

Agents
Rich output via skills
PPTX · PDF · XLSX

Connect → Perceive → Bootstrap → First Value → Accumulate → Compose → Compound ↻

Why this can't be bolted on:

Big Players, LLM Providers

No value chain from connection to compounding.
Each session isolated. No agent-to-agent feedback loop.

Agent Startups

Can execute but can't accumulate.
No Phases 5-7 (compounding).
Every run starts fresh.

Workspace AI

Trapped in one platform. Can't compose cross-platform agents.
No recursive perception.

Every layer purpose-built for knowledge accumulation, true autonomy + intelligence

Product

How Your Agent Network Grows

You don’t configure agents. The system grows them from your work — each layer builds on the last.

L1: PLATFORM DIGESTS

Auto-Created on Connect

You connected Slack. 60 seconds later, a Recap Agent ran. Gmail gets a Digest. Calendar gets a Brief. Zero configuration.

T+60 SECONDS:

“Here’s everything from #engineering, your inbox, and 3 Notion updates.” Runs every morning. Improves with every correction.

L2: SYNTHESIS AGENTS

Composer-Created

The system notices your L1 digests overlap. It creates a cross-platform agent automatically — no prompt from you.

WEEK 2:

Composer detects your recurring 1:1 pattern. Creates a Meeting Prep Agent that pulls attendee context from Slack + Gmail + Notion before every meeting.

L3: INTELLIGENCE AGENTS

Compound Insights

L1 and L2 agents feed a growing knowledge base. L3 agents surface insights no single platform could produce alone.

MONTH 2:

“Competitor X was mentioned in 3 Slack channels, 2 client emails, and a Notion brief this week. Here’s a pattern summary.” No human asked for this.

Product

Week 1 vs. Week 12

Same agent. Same workflow. Radically different output quality — because knowledge compounds.

WEEK 1
Generic digest. All channels weighted equally. Standard formatting. You correct tone and trim noise.
What the agent knows:
Platform structure. Channel names. Message content. Nothing about you.

→

WEEK 4
Knows which channels matter to you. Learned your preferred format from edits. Skips noise. Rare corrections.
What the agent knows:
Your priorities. Your edit patterns. Which sources you act on vs. ignore.

→

WEEK 12
Zero corrections needed. Anticipates your priorities before you state them. Cross-references patterns across platforms.
What the agent knows:
Your work graph. Cross-platform relationships. Historical patterns. What matters and why.

This is the gap a competitor can’t close. Starting over means starting from Week 1.

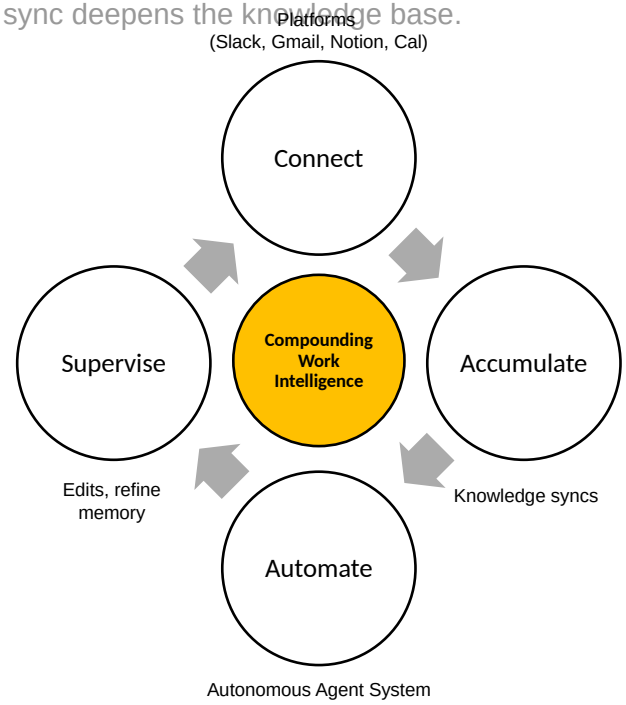
Solution: Moat

Knowledge Compounds. Switching Costs Follow.

Every agent run makes the next one better. Every edit teaches the system. Every sync deepens the knowledge base.

Three moat mechanics:

1. Accumulation: Every sync cycle deepens cross-platform understanding. Agents don't start from zero.
2. Recursion: Your edits and feedback train agent output quality. The 5th run needs fewer corrections than the 1st.
3. Irreversibility: 90 days of accumulated knowledge can't be exported or replicated. Switching means starting over.



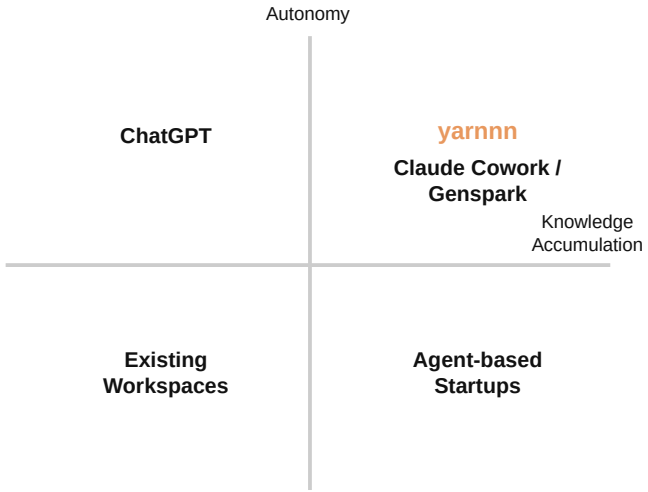
Positioning • Market • Competitive Landscape

Positioning

Not another chatbot. Not another agent framework.

The only platform where persistent agents accumulate cross-platform work knowledge for autonomous output.

	yarnnn	**Big-Players** ChatGPT/Claude	**Agent Startups** Mem.ai, Genspark	**Workspaces** Notion, Slack
Cross-platform knowledge accumulation	✓ Evolving	✘	✘	Single
Cross-platform sync	✓	✘	✘	✘
Autonomous output	✓ Scheduled	✘	✓	✘
Improves with tenure	✓ Accumulation	Evolving	✘	✘
Context-aware agent	✓	Evolving	✘	Basic



Market

Starting with solo consultants. Expanding from there.

Entry wedge: Solo consultants with recurring client obligations

TAM

$4.35B

AI productivity tools market (31% CAGR)

SAM

$1.14B

~5M solo consultants globally × $228/yr (Pro annual)

Entry SOM

$4.6M

20K consultants in first 3 years

Expansion path:

Now	**Solo consultants**	Recurring reports, clear pain, willing to pay
Next	**Founders & executives**	Investor updates, board prep, strategic briefs
Then	**Teams & ops leads**	Standups, project reports, cross-team synthesis
Scale	**All knowledge workers**	Any professional with recurring work obligations

Pricing Model

Two tiers. Knowledge depth is the lever.

Free

$0 /mo

- 2 platforms connected
- 5 sources per platform
- Sync 2x/day
- 50 orchestrator messages/mo
- 2 active agents

Pro

$19 /mo

- 4 platforms connected
- Unlimited sources
- Hourly sync
- Unlimited orchestrator messages
- Unlimited agents

Enterprise

TBD /mo

- Everything in Pro
- Team workspaces
- Priority support
- Custom integrations
- Agent-to-agent protocol

Unit economics: Target 70%+ gross margin at scale. Batched sync and cached knowledge reduce per-user API cost as context accumulates.

Yearly plan: 2 months free (17% off). Pro $190/yr.

Product Roadmap

Autonomous Agent Network (shipped) → Cross-Platform Intelligence → Agent Protocol

Each phase deepens the value chain. Shipped work compounds into the next layer of capability.

SHIPPED ✓

Autonomous Agent Network

Auto-bootstrap + Composer agents. 4 platform integrations.
Output gateway: agents produce PPTX, PDF, XLSX, charts.
Feedback substrate: edit distillation + agent self-reflection.
Workspace filesystem + cross-agent collaboration.

NEXT 6-12mo.

Cross-Platform Intelligence

Projects: multi-agent assembled deliverables (decks, reports, briefs).
Composer-driven assembly from agent contributions.
Deeper platform coverage: 365, Google Drive, Social.

VISION +12mo.

Agent-to-Agent Protocol

MCP protocol interop. External agents consume YARNNN context.
Agent-to-agent coordination across systems.
Platform play: own the work context graph.

(Pre) Seed - The Ask

$500K at $5M post-money valuation.

Use of funds:

→ Tech Lead (memory/ML)

→ GTM Lead (creator community)

→ 12 months runway

Why now:

→ ClawdBot: 17,830 stars / 24hrs

→ No one owns the context layer

→ Architecture built, need team

Positioning:

→ Application layer sill pre-mature

→ True Agent-native architecture

Priority 0: Team-Building

→ ML engineer for memory extraction

→ Community builder for creator GTM

→ Seed funds → team of 3

Betting on an agent-first future:

→ LLM readiness moving towards Agents

→ Thesis is the work economy will shift

Differentiation:

→ Cross-platform, LLM agnostic

→ Big players can't do cross data accumulations

yarnnn

yarnnn.com · Kevin Kim · kvkthecreator@gmail.com

Appendix

Kevin Kim

Founder & CEO, KVK Labs (yarnnn)

https://www.linkedin.com/in/kimseulki0727/

B.S. Industrial and Systems Engineering, Virginia Tech

+10 years designing CRM and GTM systems in FMCG.

CMO, CPO positions in Korea based startups, Ran e-commerce fashion brand for 5 years,

Full-stack application for yarnnn, (Next.js + FastAPI + Supabase), four platform integrations, unified agent architecture documented across 115+ Architecture Decision Records.

Targeting US market; execute GTM and team-building as priority next step.

kvkthecreator@gmail.com · yarnnn.com

Appendix: Value Realization Chain

What the User Experiences: Connect to Compound

Connect → Perceive → Bootstrap → First Value → Accumulate → Compose → Compound ↻

Time	What happens	What the user sees
T+0s	OAuth completes, sources auto-selected	Dashboard: “Slack connected”
T+60s	Bootstrap creates agent, first run executes	First agent appears, first output delivered
Day 2-7	Daily syncs accumulate, digest agent runs daily	Digests arrive on schedule, improving each run
Week 2	Composer heartbeat assesses substrate	Composer suggests or auto-creates cross-platform agent
Month 1	Multiple agents running, user has given feedback	Agents noticeably tailored to preferences
Month 3	L2-L3 agents producing insights from digests	System understands work at a level no fresh start replicates

Each phase’s output is the next phase’s input. A competitor starting from zero cannot replicate a tenured YARNNN instance.

Appendix: Architecture Depth

What a Well-Funded Team Can’t Shortcut

Each layer assumes the ones below it exist. 122 Architecture Decision Records document each tradeoff.

“Claude Code Online” — Output Gateway

Same capabilities model as Claude Code (SKILL.md + local tools + filesystem), but persistent, autonomous, and cloud-native. Two-filesystem architecture: capability filesystem (skills on Docker) + content filesystem (workspace per agent). Adding a new output format = adding a skill folder.

Feedback-to-Intelligence Loop

Three unified feedback rails: user edits distilled to persistent preferences, agent self-reflection after every run, Composer lifecycle coaching. Agents progress through seniority levels (new → associate → senior) with expanding duty portfolios. “Improves with tenure” is structural, not aspirational.

Autonomous Orchestration — Composer Heartbeat

A cron-triggered heartbeat assesses the entire agent workforce: substrate signals, content freshness, agent maturity, cross-agent patterns. Creates agents, pauses underperformers, dissolves unused ones, coaches via supervisor notes. No human in the loop.

Multi-Agent Project Coordination

PM agents coordinate contributor agents toward assembled deliverables. Contribution briefs steer agent focus. Quality assessment gates assembly. Work budget governors bound autonomous compute. Intent decomposition translates user goals into bounded, executable work plans.

Why this compounds into defensibility:

The Composer heartbeat can’t work without the feedback substrate. The feedback substrate can’t work without the workspace filesystem. The workspace filesystem can’t work without the perception pipeline populating it. Each architectural layer was designed knowing the layers above it would exist. A team starting today would need to discover these interdependencies through the same 122 decisions — or build a simpler system that can’t compound.

Full technical architecture brief available on request.